
Reflection Document for Cow-puter Vision-Animal Detection on Dairy Farms

Version 0

Team 21

COMPSCI 4ZP6

Dr.Mehdi Moradi

April 7, 2026

Group Members	Student Number	Contact
Xianpi Duan	400381987	duanx14@mcmaster.ca
Xiaozhao Duan	400381647	duanx15@mcmaster.ca
Zicheng Li	400359321	li1276@mcmaster.ca
Bozhi Zhang	400327039	zhanb50@mcmaster.ca
Zhanxu Ye	400390561	yez69@mcmaster.ca

Reflection Document for Cow-puter Vision-Animal Detection on Dairy Farms.....	0
Github Repo Link.....	1
Achievements.....	1
AI topline metric.....	1
Unique Features.....	1
Complexity.....	2
Challenges Solved.....	2
Decoupled Bounding-Box Overlay and Frame & Data Sync.....	2
Multi-Process Orchestration and Decoupled Components Reliability.....	2
Progress Reflection.....	2
1. Granular Behaviour Analysis (P1-FE-02 & P1-FE-04).....	2
2. Alert Management & Sorting (P1-FE-06).....	3
Highlights and lowlights.....	3
Highlights.....	3
Lowlights.....	3
What we learn.....	4
Appendix.....	5
Appendix A.....	5
P0 and P1 Progress Table.....	5

Github Repo Link

<https://github.com/duanxianpi/cowputer-vision>

Achievements

Cowputer Vision is a real-time cattle behavior monitoring and alerting platform that processes a live RTSP camera feed from a dairy barn, identifies individual cows using YOLOv12 with BoT-SORT tracking, classifies their behavior into 7 categories (walking, standing, feeding head-up/head-down, licking, drinking, lying), and surfaces actionable insights through a web dashboard.

AI topline metric

The system uses YOLOv12 (attention-centric YOLO architecture, arXiv 2502.12524), which achieves 2.1–2.5% mAP improvement over YOLOv10/YOLOv11 while maintaining real-time speeds (1.6ms inference at nano scale on T4 TensorRT). The vendored fork fixes upstream training instability and memory inefficiency issues. The model runs with multi-object tracking (BoT-SORT with Kalman filter and sparse optical flow GMC) to maintain persistent cow identities across frames.

Detection: The model achieved 98.34% precision, 97.93% recall, and a mean IoU of 91.53%, showing accurate cow detection and stable bounding box localization.

Behaviour classification: The model reached 78.62% mean precision, 81.48% mean recall, and 79.70% mean F1. Performance was solid for common behaviours such as standing, lying, and licking, which all reached F1 above 94%. More ambiguous classes like walking and unknown had lower scores and still have room for improvement.

Runtime: The system ran at an average of 73.8 FPS on the H100 GPU, which is fast enough for real-time processing.

Tracking stability: Out of 38 total tracks, 12 were flagged as ghost tracks, giving a ghost ratio of 31.58%. Most of these ghost tracks only appeared for a few frames before disappearing, so they are mainly short-lived noise rather than real tracking failures.

Unique Features

- Live monitoring dashboard: Real-time stat cards (detected cows, behavior distribution), doughnut charts, stacked area trend charts, and an alert summary panel, all updating continuously.
- Live camera with AI overlay: HLS video stream with real-time bounding boxes showing cow ID and classified behavior, with per-behavior filtering and per-cow timeline visualizations.
- Historical playback with tracking overlay: Date-picker browsing of archived video with bounding-box overlay, variable playback speed (1x–8x), and seek controls.

- Configurable alert rules: Visual JSON-Logic query builder for conditions like "if any cow is lying for more than 30 minutes, send email."
- Automated daily reports: Scheduled aggregation of behavior distribution, per-cow breakdowns, and estimated duration statistics, viewable in-app with charts.

Complexity

This is a full-stack, real-time AI-powered video surveillance system for dairy farms. It is not a single-concern app, but an integrated platform that fuses live computer vision, streaming video infrastructure, event-driven alerting, and a rich web dashboard.

Challenges Solved

Decoupled Bounding-Box Overlay and Frame & Data Sync

Unlike many computer vision projects, we chose not to embed bounding boxes directly into the video frames. This gave us a great deal of flexibility, for example, we can freely select which behaviors to highlight on the front end. However, this approach also presents challenges. Separating the bounding boxes from the frames requires us to synchronize the frames with the data during live and playback. In our system, we designed a decoupled dual-stream media server architecture that utilizes a single FFmpeg process with *-use_wallclock_as_timestamps 1* to produce two HLS outputs simultaneously: a low-latency live feed, and a codec-copy archival feed with 10-minute segments. Both outputs include the *-hls_flags program_date_time* flag, which embeds *#EXT-X-PROGRAM-DATE-TIME* (ISO-8601) tags into every HLS segment. This provides a consistent timestamp for both the live stream and each segment, allowing us to easily align tracking data with video frames during backend processing.

Multi-Process Orchestration and Decoupled Components Reliability

Our backend consists of a system of six independent processes—including a Django API server, four Python daemons, and an FFmpeg media server. This level of complexity requires the system to gracefully tolerate partial failures. Our *run.py* entry point loads the *.env* file and then launches each component as a child process. The key is that the FFmpeg media server, the Django API, and the various Python daemons are completely decoupled: if Django or any daemon crashes, recording continues unaffected because FFmpeg writes directly to disk. Subsequently, the PlaybackManager will catch up on the recording upon restart. Similarly, if a daemon crashes, it does not affect the Django API, and vice versa.

Progress Reflection

Overall Status: While the core infrastructure and high-level visualizations (P1-FE-03/05) have reached 100% completion, certain granular features remain in the optimization phase due to data dependencies and functional prioritization (check the detailed progress table of p0 and p1 at Appendix A).

1. Granular Behaviour Analysis (P1-FE-02 & P1-FE-04)

- **Progress:** 50% (Detailed data) & 80% (Single cow visualization).
- **Reflection:** The primary bottleneck for these features is the **scarcity of long-term, high-variance behavioral data**. While the ML model successfully identifies instantaneous actions (e.g., standing vs. feeding), generating "insightful" detailed reports or individual cow trend charts requires a larger historical dataset to identify meaningful patterns.

2. Alert Management & Sorting (P1-FE-06)

- **Progress:** 50%.
- **Reflection:** We have intentionally pivoted from the original requirement of sorting alerts by priority and timestamp in a complex data table.
- **Rationale:** Upon internal review, it was determined that **Priority-based sorting** currently lacks functional necessity, as the system does not yet have downstream features (like automated escalation or tiered notifications) tied to priority levels.
- **Adjustment:** To maintain a "Clean SaaS" philosophy and avoid UI clutter, we are currently displaying alerts in a simplified chronological feed, ensuring the most recent events remain the user's primary focus.

Highlights and lowlights

Highlights

- **Decoupled backend architecture:** Running all components of the backend as independent processes.
- **Runtime configurability:** The two-tier config (DB settings with env-var fallback), refreshed every few seconds by each daemon, means the system can be tuned in production without downtime.
- **The alert system is well-designed:** JSON-Logic as the condition DSL, per-cow state machines with duration tracking, deduplication windows, multi-channel dispatch, and a visual query builder on the frontend.
- **Strong test coverage for a capstone:** 220+ tests across both stacks, including integration tests for FFmpeg output and detailed daemon unit tests for state tracking, deduplication logic, and retry behavior.
- **Security:** Signed media tokens, directory traversal prevention, password reset with single-use tokens, email enumeration prevention (always returns 200), and proper JWT refresh flow.

Lowlights

- **In-memory cow state is volatile:** The EventMonitor's StateTracker holds all per-cow behavior state in memory only. If the event monitor process crashes or restarts, all duration counters reset. Cows that were "lying for 4 minutes and 59 seconds" would have their timers zeroed, potentially missing alerts.

- **Single-camera assumption:** The entire system is designed around one RTSP source. There's no multi-camera support in the models, the media server, or the frontend.
- **Low persist rate under performance constraints :** The tracker uses *track_buffer*: 60 frames and relatively low association thresholds. At ~10 FPS inference, the 60-frame buffer means a cow only gets 6 seconds before its track is dropped if it's temporarily occluded or missed. This means *cow_1* could become *cow_47* after a brief occlusion, fragmenting behavior duration tracking and undermining the accuracy of duration-based alert rules and per-cow report breakdowns.

What we learn

For this project, we learned that developing an end-to-end AI application requires more than just training a model in the real world. Our project required integrating behaviour detection and tracking, backend APIs, real-time video streaming, a web dashboard, and data storage into a single solution. We all gained experience in machine learning, backend services, and frontend visualization. This project helps us understand the challenges of building reliable end-to-end systems under practical constraints such as slow farm internet, on-premises deployment, and performance requirements. We also learned about teamwork, task coordination, and code review, which demonstrated to us the importance of effective communication and structured workflows in large technical projects.

Appendix

Appendix A

P0 and P1 Progress Table

ID	Level	Description	Progress
P0-ML-01	P0	Detect cows in a video frame	100%
P0-ML-02	P0	Classify cow behaviour (Standing, Lying Down, Feeding, Drinking)	100%
P0-ML-03	P0	Track detected cows across consecutive video frames	100%
P0-BE-01	P0	Receive video streams from cameras	100%
P0-BE-02	P0	Perform model inference on received video frames	100%
P0-BE-03	P0	Provide video streams with inference outputs to the frontend	100%
P0-BE-04	P0	Store received video frames in the local file system	100%
P0-BE-05	P0	Store video frame paths and corresponding inference outputs	100%
P0-FE-01	P0	Provide a web-based frontend requiring user authentication	100%
P0-FE-02	P0	Play video streams	100%
P0-FE-03	P0	Overlay bounding boxes and labels on video playback	100%
P0-FE-04	P0	Display a dashboard of cows' current behaviours and durations	100%
P1-BE-01	P1	Provide APIs to retrieve behaviour data with filtering	100%
P1-BE-02	P1	Provide CRUD APIs for alerts	100%
P1-BE-03	P1	Send proactive notifications for triggered alerts	100%
P1-BE-04	P1	Store alert triggering conditions and relevant data	100%
P1-FE-01	P1	Filter overlaid bounding boxes by behaviour labels	100%
P1-FE-02	P1	Display detailed behaviour data for each detected cow	50%
P1-FE-03	P1	Display aggregated behaviour visualizations for all cows	100%
P1-FE-04	P1	Display behaviour visualizations for a single cow	80%
P1-FE-05	P1	Configure alerts with custom conditions and actions	100%

P1-FE-06	P1	Display configured alerts sorted by priority and timestamp	50%
P1-FE-07	P1	Display triggered alerts with conditions and data	100%